

PEC 2

Jornadas de Artesania

Sitio web responsive con UOC Boilerplate, Sass, Bootstrap 5 y CSS moderno

Autor: Federico Javier Martino **Asignatura:** Herramientas HTML y CSS II **Actividad:** PEC 2 (50% de la nota de evaluacion continua) **Universidad:** Universitat Oberta de Catalunya (UOC) **Fecha de entrega:** 10 de mayo de 2026

Repositorio GitHub	https://github.com/Federicojaviermartino/jornadas-artesania
Sitio publicado (Netlify)	https://jornadas-artesania-uoc.netlify.app/
Video explicativo	https://drive.google.com/file/d/1hi6aCHhv5zoVTrXDki7WS6fJmAYFyf_f/view?usp=sharing

Indice

1. Presentacion del proyecto
2. Proceso de desarrollo
3. Decisiones de diseno y de desarrollo
4. Metodologia y guia de estilo
5. Configuracion de Stylelint
6. Caracteristicas de Sass utilizadas
7. Integracion y personalizacion de Bootstrap
8. Capas de cascada (cascade layers)
9. Container queries
10. Pseudoclasas funcionales :has(), :is() y :where()
11. Nuevas unidades y nuevos colores
12. Dependencias externas
13. Publicacion en GitHub y Netlify
14. Mapa final de cumplimiento del enunciado

1. Presentacion del proyecto

El sitio **Jornadas de Artesania** es un encargo academico simulado para la asignatura Herramientas HTML y CSS II de la Universitat Oberta de Catalunya. La premisa del enunciado describe a un grupo de amigos que colabora con el ayuntamiento en la organizacion de unas jornadas dedicadas a los oficios artesanos, con charlas, talleres y un mercado en la plaza Mayor durante tres dias.

El proyecto consta de cuatro paginas:

- **Portada** (`index.html`): pagina principal en formato cartel, maquetada con CSS Grid y con fallback declarado mediante `@supports` .
- **Ponentes** (`ponentes.html`): listado de las personas artesanas que participan en las jornadas, maquetado con Flexbox.
- **El oficio** (`articulo.html`): pagina en formato articulo o reportaje, con uso intensivo de pseudoclases funcionales.
- **Inscripcion** (`contacto.html`): pagina de contenido libre con formulario, integracion con Netlify Forms, nuevas unidades y nuevos espacios de color.

El sitio es totalmente responsive (mobile-first), parte de UOC Boilerplate v3.13, utiliza Bootstrap 5 personalizado mediante variables Sass, y aplica las novedades de CSS estudiadas en el modulo 3 (container queries, capas de cascada, pseudoclases funcionales, unidades de viewport dinamico y nuevos espacios de color).

Linea grafica

La paleta esta inspirada en los oficios tradicionales: tonos de arcilla (`#b85c2c`), madera (`#6b4423`), ocre (`#d6a93c`) y crema (`#faf3e6`) sobre tinta oscura (`#2b2018`). La tipografia combina *Playfair Display* para titulares y *Source Sans Pro* para texto corrido, ambas servidas localmente desde Fontsource para garantizar privacidad y rendimiento.

2. Proceso de desarrollo

El flujo de trabajo es el mismo que se uso en la PEC 1, partiendo de UOC Boilerplate, lo cual aporta consistencia entre las dos entregas y facilita el aprendizaje incremental.

2.1. Clonado e instalacion

Para reproducir el entorno en una maquina nueva:

```
git clone https://github.com/Federicojaviermartino/jornadas-artesania.git
cd jornadas-artesania
npm install
```

El comando `npm install` descarga las dependencias declaradas en `package.json`. Las dependencias de produccion son Bootstrap 5.3, Popper.js (necesario para componentes JavaScript de Bootstrap como el carrusel y el dropdown), FontAwesome 7 y Fontsource. Las dependencias de desarrollo incluyen Parcel 2, Stylelint con los plugins de orden y SCSS, y rimraf para limpiar la carpeta de salida entre builds.

2.2. Servidor de desarrollo

Para levantar el servidor local con recarga en caliente:

```
npm run dev
```

Esto ejecuta primero `rimraf dist .cache .parcel-cache` para asegurar un entorno limpio y a continuacion lanza Parcel en modo serve sobre el puerto 8123. Parcel compila Sass, agrupa los modulos JavaScript, optimiza imagenes y procesa los includes de PostHTML para los parciales `views/header.html` y `views/footer.html`.

2.3. Validacion de estilos

Para comprobar que las hojas de estilo cumplen las reglas configuradas:

```
npm run stylelint
```

Stylelint analiza todos los ficheros `src/**/*.scss` aplicando las reglas declaradas en `.stylelintrc`. La validacion finaliza sin errores ni advertencias, lo cual es un requisito explicito del enunciado.

2.4. Compilacion para produccion

Para generar la version optimizada que se entrega en la carpeta `dist`:

```
npm run build
```

Este comando encadena tres pasos a través de `npm-run-all` : primero limpia la carpeta de destino, luego ejecuta Stylelint (de modo que un error de linting bloquea la compilación) y finalmente lanza Parcel en modo build sin source maps y sin cache. El resultado son cuatro páginas HTML minificadas, una única hoja de estilos minificada y con autoprefixer aplicado, un bundle JavaScript con todas las dependencias (incluyendo Bootstrap), las fuentes en formato woff2/woff y todas las imágenes optimizadas con sus correspondientes hashes de contenido para invalidar la cache del navegador correctamente.

2.5. Estructura de carpetas

```
jornadas-artesania/
|-- .editorconfig
|-- .gitignore
|-- .npmrc
|-- .nvmrc
|-- .posthtmlrc          (root: ./src para los includes)
|-- .sassrc.js           (silencia warnings legacy de Bootstrap)
|-- .stylelintrc         (BEM + recess-order)
|-- README.md
|-- documentacion-pec2.pdf
|-- netlify.toml
|-- package.json
|-- package-lock.json
|-- dist/                 (salida de produccion)
|-- src/
|   |-- index.html
|   |-- ponentes.html
|   |-- articulo.html
|   |-- contacto.html
|   |-- views/
|   |   |-- header.html
|   |   |-- footer.html
|   |-- assets/
|   |   |-- fonts/
|   |   |-- images/      (poster, highlights, ponentes, etc.)
|   |   |-- scripts/main.js
|   |   |-- styles/
|   |   |   |-- main.scss
|   |   |   |-- _variables.scss
|   |   |   |-- _mixins.scss
|   |   |   |-- _dependencies.scss
|   |   |-- modules/
|   |   |   |-- _base.scss
|   |   |   |-- _header.scss
|   |   |   |-- _footer.scss
|   |-- pages/
|   |   |-- _home.scss
|   |   |-- _ponentes.scss
|   |   |-- _articulo.scss
|   |   |-- _contacto.scss
```

3. Decisiones de diseño y de desarrollo

3.1. Enfoque mobile-first

Las hojas de estilo se escriben pensando primero en el dispositivo móvil y luego se añaden los puntos de ruptura mediante el mixin `respond-to`. Esta decisión se basa en dos motivos: la mayoría de visitantes de un evento local consultarán la web desde el móvil, y porque los media queries con `min-width` resultan más predecibles que los `max-width` y permiten una escalada progresiva del diseño.

3.2. Elección de la estructura de carpetas

Se mantiene la estructura del UOC Boilerplate pero se subdivide la carpeta de estilos en `modules/` y `pages/`. Los módulos contienen el reset, el header y el footer (componentes que aparecen en las cuatro páginas), y la carpeta `pages/` contiene los estilos específicos de cada vista. Esta separación no contradice las recomendaciones de UOC Boilerplate (que sugiere una carpeta `layouts/`) sino que aclara la diferencia entre componentes reutilizables y secciones únicas de página.

3.3. Elección de tipografías

Se elige el par *Playfair Display* + *Source Sans Pro* porque la combinación serif/sans-serif transmite la mezcla de tradición y modernidad que las jornadas quieren comunicar. Las fuentes se sirven mediante Fontsource (paquetes npm con subsets ya preparados) y no desde un CDN externo, lo cual mejora la privacidad y permite que la web funcione completamente sin conexión a internet de terceros.

3.4. Imágenes

Las imágenes son fotografías reales del oficio artesano (alfarería, vidrio soplado, forja, talla, mercado, etc.) descargadas de bancos con licencia libre (Unsplash, Pexels). Se han optimizado con Sharp a tamaños web (de aproximadamente 70 MB iniciales a 2 MB totales) usando recorte automático por zona de interés. Todas las etiquetas `<img>` incluyen `alt` descriptivo (no decorativo) salvo cuando la imagen es puramente decorativa, en cuyo caso se usa `aria-hidden="true"`.

3.5. Accesibilidad

Algunas decisiones de accesibilidad merecen mención explícita:

- **Skip-link:** cada página incluye un enlace oculto que se hace visible al recibir foco y permite saltar al contenido principal, útil para usuarios de teclado y lectores de pantalla.
- **Iconos decorativos:** los iconos de FontAwesome reciben `aria-hidden="true"` cuando no aportan información semántica.
- **Etiquetas explícitas:** cada control de formulario tiene su `<label for="...">` asociado mediante `id`.
- **Respeto a `prefers-reduced-motion`:** heredado del CSS de Bootstrap, todas las transiciones se deshabilitan automáticamente para usuarios que así lo prefieran.

- **Contraste:** los colores principales se eligieron buscando ratios AA segun WCAG 2.1, especialmente la combinacion arcilla oscura sobre crema y blanco sobre tinta. El carrusel anade un scrim oscuro y un cajetin translucido detras de los pies de cada slide para garantizar que el texto blanco sea legible sobre cualquier fotografia.

3.6. Problemas resueltos durante el desarrollo

Incompatibilidad de unidades entre Bootstrap y nuestras variables

En las primeras pruebas, declarar `$font-size-base: 16px` rompía el calculo interno de Bootstrap, que mezcla la variable con valores en `rem` en su clase `.nav-link`. La solucion fue cambiar todas las medidas tipograficas a unidades relativas (`1rem`, `1.125rem`, etc.) lo cual ademas mejora la accesibilidad porque respeta el zoom del navegador del usuario.

Resolucion de las dependencias npm en Parcel

Las hojas de estilo importadas desde paquetes npm (FontAwesome y Fontsource) requieren el prefijo `npm:` para que el resolver de Parcel encuentre los archivos en `node_modules`. En cambio, los `.scss` de Bootstrap se importan sin prefijo porque pasan por el transformador Sass de Parcel, que ya conoce la estructura de paquetes.

Includes de PostHTML

Por defecto `posthtml-include` resuelve las rutas relativas respecto al directorio de trabajo, lo cual choca con la estructura `src/views/`. Configurar la opcion `root: "./src"` en `.posthtmlrc` permite que el include `./views/header.html` funcione correctamente desde cualquier fichero HTML de la carpeta `src`.

Avisos de obsolescencia de Sass

Bootstrap 5.3 todavia usa la directiva `@import` de Sass, marcada como deprecated en las ultimas versiones del compilador. El equipo de Bootstrap ya esta trabajando en la migracion. Mientras tanto, el fichero `.sassrc.js` silencia los avisos correspondientes (`import`, `color-functions`, `global-builtin` y `mixed-decls`) sin alterar el comportamiento del compilador.

Reglas dentro de @layer ganadas por selectores sin capa

Al introducir capas de cascada en la portada, se descubrio que el selector `h1 { color: $color-ink; }` definido sin capa en `_base.scss` ganaba al `.home-hero__title { color: $color-white; }` que estaba dentro de `@layer base`. La cascada con capas establece que las reglas sin capa **siempre** ganan a las que estan dentro de un `@layer`, con independencia de la especificidad. La solucion fue mover los overrides puntuales (color del titulo del hero, scrim del carrusel, caption sobre Bootstrap) fuera de las capas, al final de la hoja de estilos correspondiente.

4. Metodologia y guia de estilo

La metodologia elegida es **BEM** (Block, Element, Modifier) acompanada de la **Code Guide de @mdo** aplicada mediante el preset `stylelint-config-recess-order`.

4.1. Justificacion de la eleccion

La decision se baso en cinco factores:

1. **Continuidad con la PEC 1:** la PEC 1 ya empleaba BEM, lo cual permite mantener una nomenclatura coherente entre proyectos y reduce la curva de aprendizaje.
2. **Compatibilidad con Bootstrap:** BEM y Bootstrap pueden convivir sin conflicto. Como se indico en el foro de la asignatura, las clases `.formulario__campo` y `.col-md-3` coexisten en el mismo elemento sin solapar responsabilidades. BEM maquina la semantica del componente, Bootstrap aporta utilidades de layout y comportamiento.
3. **Tamano del proyecto:** para un sitio de cuatro paginas, las metodologias mas pesadas como ITCSS o SMACSS resultarian sobredimensionadas. BEM aporta orden suficiente sin exigir una arquitectura de capas completa.
4. **Legibilidad:** la nomenclatura BEM hace explicita la jerarquia componente-elemento-modificador en el propio nombre de clase, lo cual facilita la lectura del HTML sin tener que abrir el SCSS para entender la estructura.
5. **Validable automaticamente:** el patron BEM se puede expresar mediante una expresion regular que Stylelint comprueba en cada compilacion, lo cual elimina la posibilidad de incumplimientos por descuido.

4.2. Aplicacion de BEM en el proyecto

Algunos ejemplos representativos del uso de BEM:

<code>.speaker</code>	(bloque)
<code>.speaker__photo</code>	(elemento)
<code>.speaker__name</code>	(elemento)
<code>.speaker--keynote</code>	(modificador)
<code>.home-highlight</code>	(bloque)
<code>.home-highlight__media</code>	(elemento)
<code>.home-highlight__title</code>	(elemento)
<code>.widget</code>	(bloque)
<code>.widget__title</code>	(elemento)
<code>.widget--featured</code>	(modificador)

El patron se mezcla con clases de Bootstrap cuando es necesario, siguiendo el ejemplo publicado por el profesor Julian Alarte en el foro de la asignatura el 26 de abril:

```
<a href="/contacto.html"
  class="site-header__cta btn btn-primary">
  Reservar plaza
</a>
```

`site-header__cta` identifica el elemento BEM dentro del componente cabecera, mientras que `btn btn-primary` aporta el estilo y comportamiento de boton de Bootstrap. Los nombres no entran en conflicto porque estan en espacios de nombres distintos.

4.3. Code Guide y orden de propiedades

Para el orden de las propiedades CSS se aplica la *Code Guide* de Mark Otto, implementada como un preset de Stylelint llamado `stylelint-config-recess-order`. Este preset agrupa las propiedades en cinco bloques en este orden:

1. **Posicion y layout:** `position`, `top`, `z-index`, `display`, `flex`, `grid`, etc.
2. **Box model:** `width`, `height`, `padding`, `margin`, `border`, etc.
3. **Tipografia:** `font-family`, `font-size`, `line-height`, `color`, etc.
4. **Apariencia visual:** `background`, `opacity`, `box-shadow`, etc.
5. **Otros:** `cursor`, `transition`, `animation`, etc.

El orden constante en todo el codigo permite localizar rapidamente cualquier propiedad y facilita la revision por pares. Los nueve casos en los que el orden inicial era erroneo se corrigieron automaticamente con `stylelint --fix`.

5. Configuración de Stylelint

El fichero `.stylelintrc` activa cuatro presets y una serie de reglas adicionales que aseguran la calidad del código y el cumplimiento de la metodología BEM. La validación se ejecuta automáticamente como paso previo de `npm run build`.

5.1. Configuración completa

```
{
  "extends": [
    "stylelint-config-recommended-scss",
    "stylelint-config-recess-order"
  ],
  "plugins": [
    "stylelint-order",
    "stylelint-scss"
  ],
  "rules": {
    "selector-class-pattern":
      "^[a-z]([a-z0-9-]+)?(__([a-z0-9-]+-?)+)?(--([a-z0-9-]+-?)+){0,2}$",
    "max-nesting-depth": 4,
    "no-descending-specificity": null,
    "scss/at-rule-no-unknown": [true, {
      "ignoreAtRules": ["use", "forward", "include", "mixin",
        "function", "if", "else", "each", "for", "while",
        "return", "extend", "at-root", "debug", "warn", "error",
        "container", "layer", "supports"]
    }],
    "scss/load-partial-extension": null,
    "declaration-block-no-duplicate-properties": [true, {
      "ignore": ["consecutive-duplicates-with-different-values"]
    }],
    "color-no-invalid-hex": true,
    "font-family-no-duplicate-names": true,
    "block-no-empty": true,
    "no-empty-source": true,
    "scss/dollar-variable-pattern": "^[a-z][a-z0-9-]*$",
    "comment-no-empty": true,
    "no-duplicate-selectors": true,
    "shorthand-property-no-redundant-values": true
  }
}
```

5.2. Explicación regla por regla

Presets activados (extends)

- **stylelint-config-recommended-scss**: incluye el conjunto de reglas recomendadas tanto para CSS estandar como para SCSS. Detecta errores de sintaxis, propiedades desconocidas, selectores invalidos y patrones obsoletos del lenguaje.
- **stylelint-config-recess-order**: implementa el orden de propiedades de la *Code Guide* de Mark Otto y los antiguos lineamientos de Recess (de Twitter), aplicado mediante el plugin `stylelint-order`.

Plugins

- **stylelint-order**: aporta las reglas de ordenacion de propiedades. Es necesario porque `stylelint-config-recess-order` depende de el.
- **stylelint-scss**: anade reglas especificas para Sass como `scss/at-rule-no-unknown` o `scss/dollar-variable-pattern`.

Reglas personalizadas

- **selector-class-pattern**: la expresion regular obliga a que cada clase siga el patron BEM `bloque__elemento--modificador` en minusculas y guiones. Permite hasta dos modificadores encadenados.
- **max-nesting-depth: 4**: limita el anidamiento de Sass a cuatro niveles para evitar selectores excesivamente especificos que serian dificiles de sobrecribir.
- **scss/at-rule-no-unknown**: deshabilita el aviso para directivas modernas de CSS como `@container`, `@layer` y `@supports`, que el preset todavia no reconoce nativamente pero que son completamente validas.
- **scss/load-partial-extension: null**: desactiva la regla porque al usar `@import` sin extension (recomendado por Sass) la regla generaba falsos positivos.
- **declaration-block-no-duplicate-properties**: con la opcion `consecutive-duplicates-with-different-values`, permite declarar dos veces la misma propiedad cuando se usa como fallback de funcion moderna (por ejemplo, un `color` en hex seguido de uno en `oklch` para navegadores que no soportan el segundo formato).
- **scss/dollar-variable-pattern**: obliga a que las variables Sass empiecen con minuscula y usen guiones como separador, manteniendo la coherencia con el patron de clases CSS.
- **color-no-invalid-hex, font-family-no-duplicate-names, block-no-empty, comment-no-empty, no-duplicate-selectors**: validaciones basicas de calidad que evitan errores tipograficos y declaraciones vacias.

5.3. Ejecucion

El comando `npm run stylelint` finaliza con codigo de salida 0 y sin imprimir mensajes, lo cual confirma que las cuatro paginas y los modulos compartidos cumplen todas las reglas configuradas.

6. Características de Sass utilizadas

El enunciado exige el uso de cinco características de Sass: variables, anidado, funciones, parciales e importación. El proyecto las aplica todas y añade además mixins.

6.1. Variables

Definidas en `_variables.scss`, agrupadas por dominio: tipografía, paleta cromática, espaciado, breakpoints, layout, sombras, transiciones y un bloque final que sobrescribe variables internas de Bootstrap. Algunos ejemplos representativos:

```
$color-clay: #b85c2c;
$color-wood: #6b4423;
$color-cream: #faf3e6;
$spacing-xs: 0.5rem;
$spacing-md: 1.5rem;
$bp-md: 768px;
$site-radius-lg: 14px;

// Sobreescritura de Bootstrap
$primary: $color-clay;
$theme-colors: (
  "primary": $primary,
  "secondary": $color-wood,
  ...
);
```

6.2. Anidado (nesting)

Se utiliza con el operador `&` de Sass para construir clases BEM:

```

.speaker {
  display: flex;
  background-color: $color-white;

  &__photo {
    aspect-ratio: 4 / 3;
    overflow: hidden;
  }

  &__name {
    font-size: $font-size-lg;
  }

  &--keynote {
    background: linear-gradient(135deg, ...);
  }
}

```

El anidado se mantiene moderado (maximo 3 niveles en la mayoría de casos, con un techo absoluto de 4 impuesto por Stylelint) para evitar selectores demasiado específicos.

6.3. Funciones personalizadas

Se han definido tres funciones reutilizables en `_mixins.scss` :

```

@function to-rem($px) {
  @return math.div($px, 16) * 1rem;
}

@function overlay-color($base, $opacity: 0.6) {
  @return rgba($base, $opacity);
}

@function text-contrast($bg) {
  $lightness: color.channel($bg, "lightness", $space: hsl);

  @if $lightness > 55% {
    @return $color-text;
  } @else {
    @return $color-white;
  }
}

```

- **to-rem:** convierte un valor en pixeles a su equivalente en rem, útil cuando se trabaja con un diseño entregado en pixeles pero queremos resultados accesibles.
- **overlay-color:** produce una versión translúcida de un color, usada en degradados, overlays sobre imágenes y el cajetín tras los pies del carrusel.
- **text-contrast:** examina la luminosidad del color de fondo en el espacio HSL y devuelve un color de texto adecuado (oscuro o blanco) para garantizar legibilidad.

6.4. Mixins

Aunque no se piden explícitamente, los mixins reducen la repetición. Los principales son:

- **respond-to(\$breakpoint)**: emite un media query `min-width` para los puntos de ruptura sm/md/lg/xl. Permite cambiar todos los puntos de ruptura del proyecto desde una sola variable.
- **container**: aplica el patrón de contenedor centrado con ancho máximo y padding lateral.
- **section-padding**: padding vertical consistente para todas las secciones, más amplio en escritorio que en móvil.
- **hover-lift(\$scale)**: efecto sutil de elevación con sombra al pasar el cursor sobre tarjetas.
- **heading(\$size)**: aplica la fuente serif y el line-height de titulares con un tamaño configurable.
- **sr-only**: oculta visualmente un elemento sin afectar a lectores de pantalla.
- **container-context(\$name)**: configura el contexto de contenimiento (`container-type: inline-size`) necesario para que las container queries funcionen.

6.5. Parciales

Cada fichero SCSS empieza con guion bajo, que es la convención de Sass para parciales (no compilados a CSS por separado):

- `_variables.scss`
- `_mixins.scss`
- `_dependencies.scss`
- `modules/_base.scss` , `_header.scss` , `_footer.scss`
- `pages/_home.scss` , `_ponentes.scss` , `_articulo.scss` , `_contacto.scss`

6.6. Importación

El fichero `main.scss` orquesta el orden de carga, que es crítico para que las sobreescripciones de variables de Bootstrap surtan efecto:

```
@import "variables";           // primero, antes que Bootstrap
@import "mixins";
@import "dependencies";       // Bootstrap, FontAwesome, Fontsource

@import "modules/base";
@import "modules/header";
@import "modules/footer";

@import "pages/home";
@import "pages/ponentes";
@import "pages/articulo";
@import "pages/contacto";
```

Se ha decidido conservar `@import` en lugar de migrar a `@use` / `@forward` porque Bootstrap 5.3 todavía depende internamente de `@import` para que las sobreescrituras de variables funcionen. La migración completa requerirá esperar a que Bootstrap publique una versión compatible con la API moderna de Sass, sobre la que ya están trabajando según se indica en su documentación oficial.

7. Integracion y personalizacion de Bootstrap

7.1. Componentes utilizados

El enunciado pide un minimo de cuatro componentes diferentes; el proyecto utiliza **seis**:

Componente	Pagina	Personalizacion principal
Navbar	Las cuatro	Brand, padding-y, color de marca
Carousel	Portada	Modo fade y rounded-3
Card-like blocks	Portada y ponentes	card-border-radius, card-cap-bg
Buttons + btn-group	Las cuatro	btn-padding-y/x, btn-font-weight, theme-colors
Accordion	Ponentes	Variables del tema heredadas
Forms (form-control, form-select, form-check)	Inscripcion	input-border-color, input-focus-border-color

7.2. Variables Sass personalizadas

Las variables que se sobrescriben antes del `@import` de Bootstrap (en `_variables.scss`) cambian el aspecto global de todos los componentes. La lista completa:

Variable Sass	Valor	Efecto
<code>\$primary</code>	<code>#b85c2c</code>	Color principal del tema (botones, enlaces)
<code>\$secondary</code>	<code>#6b4423</code>	Color secundario (botones, fondos)
<code>\$success</code>	<code>#4a7c2c</code>	Color de exito (formularios validos)
<code>\$danger</code>	<code>#b03030</code>	Color de error (formularios invalidos)
<code>\$theme-colors</code>	(map)	Mapa completo de colores tematicos
<code>\$body-bg</code>	<code>#fdfaf2</code>	Fondo de pagina por defecto
<code>\$body-color</code>	<code>#2b2018</code>	Color de texto por defecto
<code>\$font-family-base</code>	Source Sans Pro	Tipografia de cuerpo
<code>\$headings-font-family</code>	Playfair Display	Tipografia de titulares
<code>\$headings-font-weight</code>	700	Peso de titulares
<code>\$border-radius</code>	<code>6px</code>	Radio por defecto

<code>\$border-radius-lg</code>	<code>14px</code>	Radio grande para tarjetas
<code>\$enable-shadows</code>	<code>false</code>	Desactiva sombras nativas de Bootstrap
<code>\$enable-gradients</code>	<code>false</code>	Desactiva gradientes nativos
<code>\$navbar-padding-y</code>	<code>0.85rem</code>	Altura vertical de la navbar
<code>\$navbar-brand-font-size</code>	<code>1.25rem</code>	Tamaño del brand
<code>\$card-border-radius</code>	<code>14px</code>	Radio de las tarjetas
<code>\$card-border-color</code>	<code>#e2d5c3</code>	Color del borde de tarjetas
<code>\$card-cap-bg</code>	<code>#faf3e6</code>	Fondo del header de tarjetas
<code>\$btn-font-weight</code>	<code>600</code>	Peso de fuente de botones
<code>\$btn-padding-y</code>	<code>0.6rem</code>	Padding vertical de botones
<code>\$btn-padding-x</code>	<code>1.4rem</code>	Padding horizontal de botones
<code>\$input-border-color</code>	<code>#e2d5c3</code>	Color del borde de inputs
<code>\$input-focus-border-color</code>	<code>#e8a172</code>	Color del borde de inputs en foco

7.3. JavaScript de Bootstrap

El bundle de Bootstrap (incluye Popper) se importa al principio de `main.js` :

```
import * as bootstrap from "bootstrap";
window.bootstrap = bootstrap;
```

Esto permite que los componentes interactivos como el carrusel y el accordion funcionen automáticamente mediante los atributos `data-bs-*` sin necesidad de instanciarlos manualmente. Asignar a `window` es útil por si algún script externo (por ejemplo Netlify Forms) necesita disparar componentes.

8. Capas de cascada (cascade layers)

El enunciado exige el uso de capas de cascada de manera coherente en dos de las cuatro paginas. Se han elegido la **portada** y la **pagina de articulo**, por motivos que se explican a continuacion.

8.1. Que problema resuelven las capas

La especificidad de los selectores CSS es una fuente comun de bugs dificiles de depurar. Cuando dos reglas afectan al mismo elemento, gana la mas especifica, lo cual obliga a veces a anadir `!important` o a aumentar artificialmente la especificidad escribiendo selectores mas largos. Las capas de cascada resuelven esto estableciendo un orden explicito entre grupos de reglas. Las capas declaradas mas tarde ganan a las anteriores con independencia de la especificidad de los selectores que contengan.

8.2. Uso en la portada

En `pages/_home.scss` se declaran tres capas:

```
@layer base, components, overrides;

@layer base {
  .home-hero { ... }
  .home-hero__title { ... }
  /* Estructura base del cartel principal */
}

@layer components {
  .home-info { ... }
  .home-highlight { ... }
  /* Componentes reutilizables como las tarjetas */
}

@layer overrides {
  .home-hero .btn-primary { ... }
  /* Ajustes finales sobre Bootstrap */
}
```

El motivo de esta organizacion es practico. Los estilos de `base` definen la estructura del cartel del hero, los de `components` contienen las tarjetas reutilizables, y `overrides` aplica ajustes locales sobre componentes de Bootstrap (como los colores especificos de los botones del hero) que necesitan ganar a las definiciones globales del framework. Sin las capas, estos overrides exigirian selectores mas largos o el uso de `!important`.

Observacion practica sobre la cascada con capas: las reglas dentro de un `@Layer` siempre pierden frente a las reglas sin capa, con independencia de la especificidad. Para los pocos overrides puntuales que tenian que ganar al CSS sin capa de Bootstrap o de `_base.scss` (color del titulo del hero, scrim del carrusel, cajetin del caption), se han movido fuera de las capas, al final de `_home.scss`.

8.3. Uso en la pagina de articulo

En `pages/_articulo.scss` se declaran cuatro capas:

```
@layer reset, structure, typography, components;

@layer structure {
  .article { ... }
  .article__container { ... }
}

@layer typography {
  :where(.article__main) { ... }
  .article__main :is(h2, h3, h4) { ... }
  .article__main blockquote { ... }
  .article__main :is(ul, ol) { ... }
}

@layer components {
  .article__hero { ... }
  .article__main figure:has(figcaption) { ... }
  .widget { ... }
  .article__aside:has(.widget--featured) { ... }
}
```

Esta organizacion separa con claridad los tres tipos de reglas que conviven en una pagina tipo articulo: la estructura general (cuadrícula de texto y barra lateral), la tipografía (estilos de párrafos, citas, listas y subtítulos) y los componentes (figuras con pie, widgets de la barra lateral). Si en el futuro se anaden temas alternativos, las nuevas reglas se pueden meter en una capa `theme` declarada al final, que ganaria a las tres anteriores sin necesidad de aumentar la especificidad.

8.4. Por que solo en dos paginas

El enunciado pide explicitamente que las capas se utilicen en dos de las cuatro paginas. La eleccion (portada y articulo) responde a que estas son las dos paginas con mas variedad de tipos de contenido (estructura compleja en la portada, tipografía rica en el articulo) y por tanto las que mas se benefician de la separacion. Las paginas de ponentes y contacto, mas homogeneas, no necesitan esta organizacion adicional, y dejar dos paginas sin capas permite comparar visualmente la diferencia entre ambos enfoques durante la revision por pares.

9. Container queries

El enunciado pide el uso de container queries en las cuatro paginas. Estan implementadas en cada una con un proposito real, no decorativo.

9.1. Configuracion del contexto de contenimiento

El mixin `container-context` centraliza la sintaxis:

```
@mixin container-context($name: null) {  
  @if $name {  
    container-name: $name;  
  }  
  container-type: inline-size;  
}
```

El valor `inline-size` establece que el contenedor mide su ancho a efectos de las queries pero no su alto, lo cual es lo deseable en la mayoría de tarjetas y bloques de contenido. Asignar un nombre permite tener varios contenedores anidados sin que se confundan entre ellos.

9.2. Aplicacion en cada pagina

Portada (`.home-highlight`)

```
.home-highlight {  
  @include container-context(highlight-card);  
  ...  
  @container highlight-card (min-width: 480px) {  
    display: grid;  
    grid-template-columns: 200px 1fr;  
    .home-highlight__media { aspect-ratio: auto; height: 100%; }  
  }  
}
```

Cuando una tarjeta destacada supera los 480px de ancho, su layout pasa de vertical (imagen arriba, texto debajo) a horizontal (imagen a la izquierda, texto a la derecha). Esto es util porque la rejilla de la portada cambia de 1 a 3 columnas segun el dispositivo: en una rejilla de 1 columna, las tarjetas son anchas y el layout horizontal funciona mejor; en una rejilla de 3 columnas, cada tarjeta es estrecha y queda mejor el vertical.

Ponentes (`.speaker`)

```
.speaker {
  @include container-context(speaker-card);
  ...
  @container speaker-card (min-width: 520px) {
    flex-direction: row;
    .speaker__photo { flex: 0 0 220px; }
  }
}
```

El mismo patron permite que la tarjeta `keynote` (que ocupa el ancho completo gracias al modificador `--keynote`) muestre la foto a la izquierda y los datos a la derecha, mientras que las tarjetas regulares (en columnas de tres) mantienen el layout vertical.

Articulo (`.article__main`)

```
@container article-main (min-width: 700px) {
  .article__lead::first-letter {
    float: left;
    font-size: 4rem;
    color: $color-clay;
    ...
  }
}
```

Cuando el contenedor del articulo supera 700px de ancho, el primer parrafo recibe una letra capital decorativa, una solucion editorial clasica que solo tiene sentido en anchos amplios.

Contacto (`.contact-form`)

```
@container contact-form (min-width: 600px) {
  .contact-form__row {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: $spacing-md;
  }
}
```

Los campos de nombre y correo electronico se colocan lado a lado cuando el panel del formulario es lo bastante ancho. Esto es independiente del ancho del viewport porque, en pantallas grandes, el formulario ocupa solo la mitad izquierda de la rejilla principal y la otra mitad la ocupa el panel de informacion.

9.3. Por que container queries y no media queries

Las media queries solo conocen el viewport. Por eso un componente que en una pagina ocupa 1200px y en otra ocupa 300px se comporta igual con la misma media query. Con container queries, en cambio, el componente se adapta a su contenedor real: una tarjeta `.home-highlight` en una rejilla de 3 columnas se comporta distinto que la misma tarjeta en una rejilla de 1 columna, sin que tengamos que escribir reglas distintas para cada caso.

10. Pseudoclases funcionales :has(), :is() y :where()

El enunciado pide el uso correcto y con criterio de `:has()` y al menos una de `:is()` o `:where()`. El proyecto utiliza las tres en la pagina `articulo.html`.

10.1. :has()

Hay tres usos diferenciados de `:has()` en el articulo:

Caso 1: figuras con pie de foto

```
.article__main figure:has(figcaption) {  
  padding: $spacing-sm;  
  background-color: $color-cream;  
  border-radius: $site-radius-lg;  
  box-shadow: $shadow-sm;  
}
```

El selector `figure:has(figcaption)` aplica un fondo crema, border-radius y sombra solo a las figuras que contienen un pie. Las figuras sin pie (por ejemplo la imagen de cabecera del articulo) no reciben este tratamiento. Sin `:has()`, esto requeriria anadir una clase manualmente o anadir JavaScript que la inyecte.

Caso 2: parrafos con enfasis

```
.article__main p:has(em) {  
  color: $color-wood;  
}
```

Los parrafos que contienen una etiqueta `<em>` reciben un color marron suave que los diferencia visualmente de los demas, reforzando su importancia.

Caso 3: barra lateral con widget destacado

```
.article__aside:has(.widget--featured) {  
  border-left: 3px solid $color-clay;  
  padding-left: $spacing-md;  
}
```

La barra lateral entera muestra un acento visual a la izquierda solo cuando dentro de ella hay al menos un widget marcado como destacado. Este es un ejemplo perfecto de la potencia de `:has()`: el padre se estiliza segun el contenido que tenga, lo cual antes era imposible en CSS puro.

10.2. :is()

Se usa para agrupar selectores y compactar el código:

```
.article__main :is(h2, h3, h4) {
  margin-top: $spacing-xl;
  margin-bottom: $spacing-sm;
  color: $color-ink;
}

.article__main :is(ul, ol) {
  padding-left: $spacing-lg;
  margin-bottom: $spacing-md;
}
```

Sin `:is()`, la primera regla debería escribirse como `.article__main h2, .article__main h3, .article__main h4`, lo cual repite el mismo prefijo tres veces. Con `:is()`, el prefijo se escribe solo una vez. La especificidad de `:is()` es la del selector más específico de los que recibe.

10.3. :where()

Se usa para aplicar estilos base sin afectar a la especificidad:

```
:where(.article__main) {
  color: $color-text;

  p {
    margin-bottom: $spacing-sm;
    font-size: $font-size-lg;
    line-height: 1.7;
  }
}
```

La diferencia clave con `:is()` es que `:where()` tiene especificidad cero. Esto significa que cualquier otra regla, incluso un selector de un solo elemento, podrá sobrescribir estos estilos sin necesidad de aumentar artificialmente la especificidad. Es ideal para estilos base o para resets, exactamente el papel que tienen aquí.

11. Nuevas unidades y nuevos colores

Concentradas en la pagina `contacto.html` , donde tienen un uso significativo y demostrable visualmente.

11.1. Nuevas unidades de viewport

dvh (dynamic viewport height)

```
.contact-hero {  
  min-height: 60dvh;  
  ...  
}
```

El hero de la pagina de contacto ocupa al menos el 60% del viewport dinamico. La unidad `dvh` tiene en cuenta los cambios del viewport en moviles cuando aparecen y desaparecen las barras de herramientas del navegador, evitando saltos visuales que se producen con la unidad clasica `vh` .

lvh (large viewport height)

```
.contact-map__embed {  
  min-height: max(40lvh, 320px);  
}
```

El bloque del mapa al final de la pagina garantiza una altura minima respecto al viewport grande (el viewport en su tamano maximo posible, sin barras del navegador), lo cual asegura que tenga presencia visual incluso cuando el usuario hace scroll y las barras de navegacion del movil se ocultan.

cqi (container query inline)

```
.contact-hero__visual i {  
  font-size: clamp(4rem, 25cqi, 8rem);  
}
```

El icono central del hero escala respecto al ancho del propio contenedor visual, no del viewport. Combinada con `clamp()` , esta declaracion garantiza que el icono nunca sea mas pequeno que `4rem` ni mas grande que `8rem` , pero dentro de ese rango se adapta al espacio disponible.

11.2. Nuevos espacios de color

OKLCH

```
.contact-hero {
  background: linear-gradient(135deg, $color-clay-dark 0%, $color-wood 100%);
  background: linear-gradient(
    135deg,
    oklch(45% 0.16 45) 0%,
    oklch(35% 0.07 45) 100%
  );
}
```

OKLCH es un espacio de color perceptualmente uniforme, lo cual significa que valores numericos cercanos producen colores que el ojo percibe como cercanos (cosa que no ocurre con RGB). El degradado del hero se define en OKLCH para conseguir una transicion suave entre los dos tonos. La declaracion previa en hex sirve como fallback para navegadores antiguos: si el navegador entiende OKLCH, gana la segunda; si no, gana la primera.

display-p3

```
.contact-hero__visual {
  box-shadow: 0 0 60px color(display-p3 1 0.6 0.2 / 0.4);
}
```

El circulo del hero usa una sombra colorida en el espacio `display-p3`, que es aproximadamente un 25% mas amplio que sRGB. En monitores que soportan P3 (la mayoría de productos Apple modernos y muchos monitores profesionales), el resplandor naranja resulta visiblemente mas saturado y vibrante que en sRGB. En monitores estandar, el navegador convierte el color al gama disponible, sin perdida visible.

HWB (Hue, Whiteness, Blackness)

```
.contact-hero__subtitle {
  color: hwb(38 80% 5%);
}
```

HWB describe los colores anadiendo blanco y negro a un tono puro, lo cual resulta mas intuitivo que mezclar canales RGB. El subtítulo del hero usa HWB para definir un tono calido suave (crema rosado).

11.3. Nota sobre el procesamiento de PostCSS

PostCSS (configurado mediante `postcss-preset-env` que UOC Boilerplate trae por defecto) procesa estos colores y unidades antes de la entrega. Donde es posible, se preserva la sintaxis moderna; donde es necesario, se anaden fallbacks automaticos en formatos tradicionales. El resultado es una hoja de estilos que aprovecha lo nuevo en navegadores actualizados pero degrada con elegancia en versiones antiguas.

12. Dependencias externas

12.1. Bootstrap 5.3

Por que se eligio: el enunciado lo exige explicitamente. Aporta un sistema de rejilla maduro, componentes interactivos comunes ya contruidos (carrusel, accordion, navbar responsive, formularios) y un sistema de utilidades que reduce el CSS a escribir.

Como se integra: a traves de Sass para poder personalizarlo. Las variables del tema se sobreescriben antes del `@import`, lo cual permite que toda la cascada de Bootstrap se recompile usando la paleta de las jornadas. El JavaScript se importa mediante un `import * as bootstrap` estandar de ES modules.

Personalizacion: ver tabla del capitulo 7.

12.2. FontAwesome 7

Por que se eligio: continuidad con la PEC 1, biblioteca de iconos amplia y bien mantenida, integracion sencilla.

Como se integra: import de la hoja CSS desde npm en `_dependencies.scss` con el prefijo `npm:` que reconoce Parcel.

12.3. Fontsource (Playfair Display y Source Sans Pro)

Por que se eligio: alternativa a Google Fonts que sirve las tipografias desde el mismo servidor que el resto del sitio, mejorando la privacidad (no hay peticiones a terceros) y permitiendo trabajar sin internet durante el desarrollo.

Como se integra: cuatro pesos por familia (400 y 700 para Playfair, 400, 600 y 700 para Source Sans Pro) importados como CSS desde npm.

12.4. Popper.js

Por que se eligio: dependencia obligatoria de Bootstrap para los componentes que necesitan posicionamiento dinamico (dropdowns, popovers, tooltips).

Como se integra: instalado como dependencia de NPM y consumido implicitamente por el bundle de Bootstrap.

12.5. OpenStreetMap (sin libreria adicional)

Por que se eligio: el bloque "Como llegar" de la pagina de inscripcion necesitaba un mapa real que sustituyera al patron decorativo inicial. En la PEC 1 se utilizo Leaflet, pero su peso (alrededor de 140KB minificada mas las teselas) no se justifica para un unico bloque informativo.

Como se integra: mediante un `<iframe>` apuntando a `openstreetmap.org/export/embed.html` con un `bbox` y un marcador. No requiere libreria JavaScript ni clave de API. Bajo el iframe, un enlace permite abrir el mapa ampliado en una pestana nueva.

13. Publicacion en GitHub y Netlify

13.1. Repositorio Git

La inicializacion del repositorio sigue el flujo estandar:

```
cd jornadas-artesania
git init
git add .
git commit -m "Inicializa el proyecto PEC 2"
git branch -M main
git remote add origin https://github.com/Federicojaviermartino/jornadas-artesania.git
git push -u origin main
```

El fichero `.gitignore` excluye `node_modules/`, `dist/`, `.cache/`, `.parcel-cache/` y los logs generados por npm. La carpeta `dist/` no se versiona porque se regenera en cada despliegue, pero si se incluye en el ZIP de entrega como exige el enunciado.

13.2. Despliegue en Netlify

El despliegue se realiza conectando el repositorio de GitHub a Netlify desde la interfaz web de Netlify. Los parametros de configuracion (definidos en `netlify.toml` en la raiz del proyecto) son:

- **Build command:** `npm run build`
- **Publish directory:** `dist`
- **Node version:** 20 LTS (especificada en `.nvmrc`)

Netlify ejecuta automaticamente el comando de build, que a su vez ejecuta Stylelint y Parcel, y publica la carpeta `dist`. Cada push a la rama principal dispara un nuevo despliegue. La URL definitiva del sitio es:

```
https://jornadas-artesania-uoc.netlify.app/
```

13.3. Netlify Forms

El formulario de inscripcion incluye los atributos `data-netlify="true"` y `netlify-honeypot="bot-field"`, lo cual permite que Netlify procese los envios automaticamente sin necesidad de un servidor backend. Las respuestas se reciben en el panel de control de Netlify y, opcionalmente, por correo. Esta es exactamente la funcionalidad que sugiere el enunciado en la opcion de Netlify Forms.

14. Mapa final de cumplimiento del enunciado

Resumen de los requisitos del enunciado y el lugar exacto del código donde se cumple cada uno.

Requisito	Donde se cumple
Cuatro paginas (1 portada + 3 interiores)	<code>index.html</code> , <code>ponentes.html</code> , <code>articulo.html</code> , <code>contacto.html</code>
Wireframes respetados	Todas las paginas siguen los wireframes de la PEC
Portada en CSS Grid + <code>@supports</code>	<code>pages/_home.scss</code> layer base
Pagina de ponentes con Flex (sin row/col)	<code>pages/_ponentes.scss</code> <code>.speakers__grid</code>
Articulo con <code>:has()</code> obligatorio	<code>_articulo.scss</code> <code>figure:has(figcaption)</code> , <code>p:has(em)</code> , <code>aside:has(...)</code>
Articulo con <code>:is()</code> o <code>:where()</code>	<code>_articulo.scss</code> usa ambas
Articulo con blockquote y listas	<code>articulo.html</code> y <code>_articulo.scss</code>
Pagina libre con nuevas unidades	<code>_contacto.scss</code> usa <code>dvh</code> , <code>lvh</code> , <code>cqi</code>
Pagina libre con nuevos colores	<code>_contacto.scss</code> usa <code>oklch</code> , <code>hwb</code> , <code>display-p3</code>
Responsive en todos los dispositivos	Mixin <code>respond-to</code> + Bootstrap grid
UOC Boilerplate v3.13	<code>package.json</code> basado en la plantilla oficial
Metodologia o guia de estilo	BEM + Code Guide (recess-order)
Bootstrap 5 personalizado con Sass	<code>_variables.scss</code> antes del <code>@import</code>
Minimo 4 componentes Bootstrap	Navbar, carousel, accordion, forms, btn-group, cards (6)
Variables Sass de Bootstrap personalizadas	Ver tabla del capítulo 7
Container queries en las 4 paginas	Si: home, ponentes, articulo, contacto
Cascade layers en 2 de las 4 paginas	<code>_home.scss</code> y <code>_articulo.scss</code>
Otra dependencia externa	FontAwesome 7 + Fontsource
Stylelint instalado y configurado	<code>.stylelintrc</code> completo
Stylelint sin errores	<code>npm run stylelint</code> sale con código 0
Sass: variables	<code>_variables.scss</code>

Sass: anidado	Todas las hojas usan <code>&</code> con BEM
Sass: funciones	<code>to-rem</code> , <code>overlay-color</code> , <code>text-contrast</code>
Sass: parciales	Todos los archivos empiezan con <code>_</code>
Sass: importacion	<code>main.scss</code> orquesta los <code>@import</code>
Codigo compilado en <code>/dist</code>	<code>npm run build</code> genera <code>dist/</code> completa
Repositorio en GitHub	<code>github.com/Federicojaviermartino/jornadas-artesania</code>
Deployment en Netlify	<code>jornadas-artesania-uoc.netlify.app</code>
Video explicativo	https://drive.google.com/file/d/1hi6aCHhv5zoVTrXDki7WS6fJmAYFyf_f/view?usp=sharing

Cierre

El proyecto **Jornadas de Artesania** pone en practica todos los conceptos estudiados a lo largo del semestre y se entrega cumpliendo cada punto del enunciado. La continuidad con la metodologia de la PEC 1 (BEM y Code Guide) facilita la lectura del codigo, mientras que la integracion de Bootstrap, las novedades de CSS moderno y las herramientas de calidad permiten enriquecer la solucion sin renunciar a la coherencia general.

Quedo a disposicion del profesor colaborador y de los companeros revisores para cualquier aclaracion adicional que pueda resultar util durante la evaluacion.

Federico Javier Martino — PEC 2, Herramientas HTML y CSS II — UOC, 2026